
ECE550 Post-silicon Validation

– Final Project

Final Project

- Due 6/13th, last day of class
- What is next?
 - Adding MP support.
 - Think Parallel Tasks.

New “MP” Builder Template

- New builder template provides base for startup up multiple tasks
- Allows building code for multiple Cores/Threads Under Test
- Allocates shared memory regions as well as specific regions per thread.
- Allocates a “comm” area for sharing memory between threads.
- Utilizes Unix fork for creating child processes
 - (see extra credit for thread binding).

Adding Multiple processing support

- Adding the following instructions
- add the following new instruction build routines
 - build_xadd(of the following forms) r/m{8,16,32}, {r8, r16, r32} (Intel syntax)
 - for the above, randomize with/without a lock prefix
 - build_xchg(of the following forms) r/m{8,16,32}, {r8, r16, r32} (Intel syntax)
 - for the above, randomize with/without a lock prefix
 - build_mfence(add instruction for mfence)
 - build_sfence(add instruction for sfence)
 - build_lfence(add instruction for lfence)

New Parameters

- Add New parameters
- nthreads: Add a parameter to your code to pass in the number of threads to generate and execute.
- logfilename: Add a parameter to dump out a log file on instruction type, reg, memory, displacement,

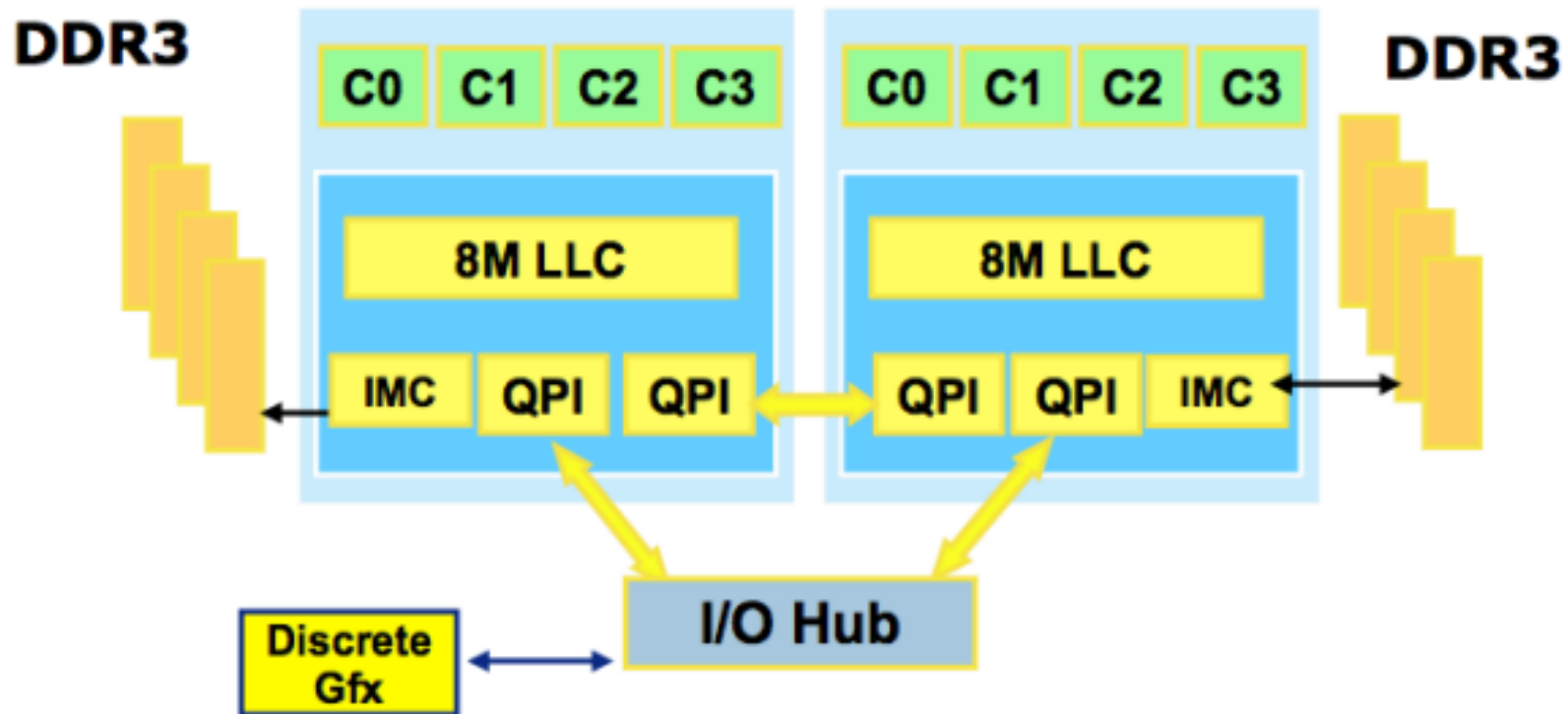
Multiple Process...

- `bash-3.2$ ./encodeit 0 20 2`
- starting seed = 0
- Number of instructions = 20
- nthreads = 2
- T0 next_ptr=0x10004d000
- child T15805 started:
- T0 fork
- building instructions for T0
- next ptr is now 0x10004d010
- next ptr is now 0x10004d01c
- next ptr is now 0x10004d022
- next ptr is now 0x10004d028
- next ptr is now 0x10004d02b
- next ptr is now 0x10004d02f
- T0 generation program complete, instructions generated: 7
- T1 next_ptr=0x100050000
- child T15806 started:
- T1 fork
- building instructions for T1
- next ptr is now 0x100050010
- next ptr is now 0x10005001c
- next ptr is now 0x100050022
- next ptr is now 0x100050028
- next ptr is now 0x10005002b
- next ptr is now 0x10005002f
- T1 generation program complete, instructions generated: 7

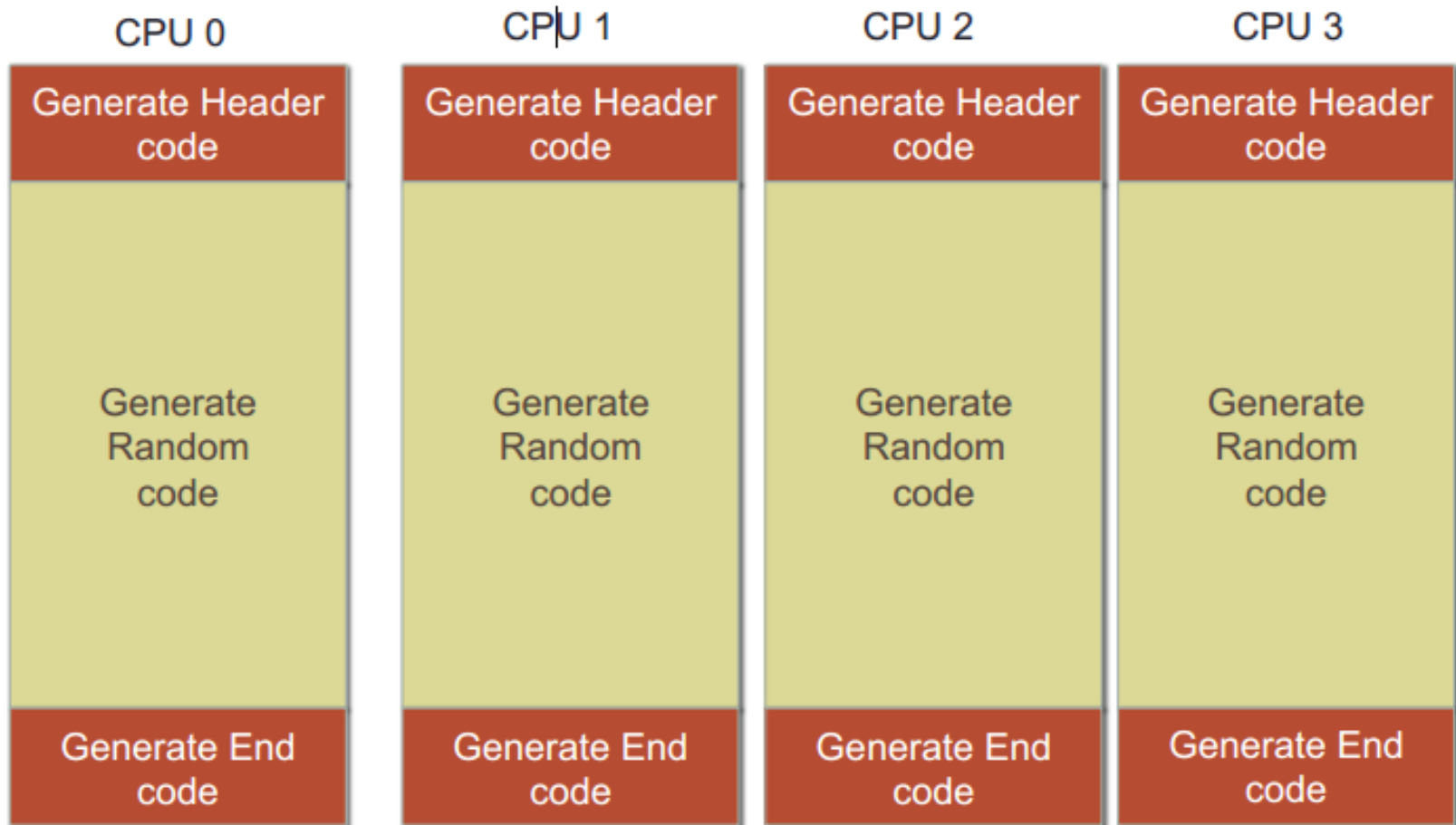
Memory barriers

- A memory barrier primitive ensures that the operations placed before the primitive are finished before starting the operations placed after the primitive.
- Thus, a memory barrier is like a firewall that cannot be passed by an assembly language instruction.
- In the **80x86 processors**, the following kinds of assembly language **instructions** are said to be "**serializing**" because they act as memory barriers:
 - All instructions that operate on **I/O ports**
 - All instructions **prefixed by the lock byte**
 - All instructions that **write into control registers, system registers, or debug registers** (for instance, cli and sti)
 - The **lfence** , **sfence** , and **mfence assembly language instructions**, which have been introduced in the Pentium 4 microprocessor to efficiently implement read memory barriers, write memory barriers, and read-write memory barriers, respectively.
 - A few **special assembly language instructions**; among them, the iret instruction that terminates an interrupt or exception handler

Uncore Micro-Architecture Coverage (i7 Nehalem), Xeon 5500



MP Generation



Binding a process to a particular CPU

```
#include <sched.h>
```

```
int sched_setaffinity(pid_t pid, size_t cpusetsize,  
    cpu_set_t *mask);
```

```
int sched_getaffinity(pid_t pid, size_t cpusetsize,  
    cpu_set_t *mask);
```

http://linux.die.net/man/2/sched_setaffinity

What are we missing

- Synchronization to start
- Synchronization at the end

Extra Credit

- Insert a Synchronization Barrier into the add_header and add_end iCode
- The barrier forces all processes to wait until all processes reach the barrier and then releases of the processes..
- i.e. force all generated code threads to wait until all of the generated code sequences have arrived at the barrier.
- Example: See D Patterson, Computer Architecture, 3rd addition, barrier synchronization.

References

- Intel SDM 8.3, Serialization instructions
- D Patterson, Computer Architecture, 3rd addition, barrier synchronization.
- Links
- sched_setaffinity:
- http://linux.die.net/man/2/sched_setaffinity
- Kernel Synchronization:
<http://linux.linti.unlp.edu.ar/images/6/63/Ulk3-capitulo5.pdf>